Can a Judge Choose Where to Branch? Prefix Selection for Tree Rollouts in GRPO on MATH

DC-Jev-RL project, Part III-C (version 4)

October 2026

Correction notice. Versions 1.0–3.0 of this paper ("Tree-Jev-GRPO") are withdrawn. Their results were not produced by any experiment. The pruning speed-ups, KV-cache reuse rates, per-node latencies and theorem-proving results (MiniF2F-Lean4, ProofNet) are numbers written into a script that contains no code that loads, trains or evaluates a model. This version studies tree rollouts on MATH only, with measured experiments, and tests the pruning rule of v3 directly. The withdrawn files are kept in the repository under `withdrawn/`, and `CORRECTION.md` lists what was wrong.

Abstract

Tree rollouts sample a few prefixes per prompt and branch from them, so that the branches share the prefix tokens. A judge that recognises a promising partial solution could choose where to branch. We test this with Qwen3.5-4B-Base on MATH. Offline, Kev-4B, an autoregressive judge of the same size and the policy's own log-probability all predict across problems whether a 128-token prefix will end correct (AUROC 0.67–0.77), but separate one problem's prefixes only weakly (0.59–0.64); keeping the two of four prefixes Kev-4B ranks highest raises the share of correct continuations by +4.6 points, the policy's log-probability by +5.2. In GRPO with tree rollouts (four roots of 128 tokens, two of them continued four times each, 200 steps, two seeds), Kev-4B's choice of roots raised the share of correct leaves by +1.4 points [0.7, 2.1], a random choice by +0.2, and the final policies are as accurate as with random roots on four of five benchmarks (GSM8K: +1.0 [0.2, 1.9]). Both tree conditions end above flat GRPO in the seed mean (MATH-500 +1.6 and +2.0 points, AIME24 +5.7 and +4.9), but each with one seed well above flat sampling and one at its level, so whether the tree shape helps is open. One seed with Kev-4B's choice came to repeat its solution after the answer, which the verifier's reward does not penalise. The pruning rule of the withdrawn earlier versions, prune a prefix when the judge's $p(\text{yes}) < 0.3$, prunes 1.6% of the prefixes. This version replaces earlier versions whose results were not produced by experiments. Code, logs and every number in this paper are in the repository.

1 Introduction

Sampling a group of complete solutions per prompt, as GRPO does, spends most of its tokens on solutions that share nothing. Tree-structured rollouts [1] sample a few prefixes and branch from them. This shares the prefix tokens and places the comparison inside the group at the point where solutions diverge. A further step is to choose which prefixes to branch from. A judge that can recognise a promising partial solution could steer the samples towards the parts of the tree where the policy can succeed. That would give GRPO more groups with mixed outcomes, the only groups with a learning signal.

We test both halves of that idea. First, offline: can Kev-4B, the AR judge of the same size, or the policy's own log-probability tell from the first 128, 256 or 512 tokens whether a solution will end correct? And does keeping the prefixes they rank highest raise the success rate of what is continued? Second, in training: does GRPO with judge-selected prefixes learn better than with randomly selected ones, at the same tree shape and token budget?

Correction. Versions 1–3 of this paper ("Tree-Jev-GRPO") reported pruning speed-ups, KV-cache reuse rates, theorem-proving results (MiniF2F, ProofNet) and per-node latencies that were not produced by any experiment (see the notice above and `CORRECTION.md`). This version studies tree rollouts on MATH only, with the measurements above. It also tests the pruning rule of v3, which pruned a prefix when the judge's $p(\text{yes})$ was below 0.3.

2 Related work

TreeRL [1] uses on-policy tree search with entropy-guided branching for LLM RL. Process reward models score partial solutions [2, 4]. Math-Shepherd builds its labels by Monte-Carlo continuation, which is how our prefix study measures a prefix's value. Search guided by a verifier at test time [3] spends compute where a judge sees promise. Selecting training samples by a learned score changes what the gradient estimates. That is the cost side of the trade-off we measure.

3 Judging unfinished solutions

Tree rollouts [1] sample several prefixes of a solution, continue some of them and share each prefix among the continuations. A judge that can tell early which prefixes lead to correct answers could decide which ones to continue. This saves the cost of continuing dead ends and concentrates samples where the policy can succeed. We first measure offline whether a 4B judge can make that call, then train with it.

Prefix study. We use GRPO's validation problems (500 MATH training problems held out from training, §2 of Part II). The base policy samples 4 solutions per problem as in GRPO (temperature 1, the training prompt). Every solution is cut at 128, 256 and 512 tokens if it runs that long. Each prefix gets 4 continuations to the training length (1024 tokens in all), and the verifier checks them. The fraction of a prefix's continuations that end correct is its Monte-Carlo value. The signals are:

- Kev-4B's $p(\text{yes})$ for *on track* ("Is this unfinished solution on track to reach the correct final answer. . . ?");
- Kev-4B's $p(\text{yes})$ for Part II's *correct* question, with the prefix as the proposed solution;
- the AR judge's $p(\text{yes})$ for *on track*;
- the policy's own mean token log-probability of the prefix, which costs nothing.

For each cut length we report each signal's AUROC for a continuation ending correct. It is reported pooled and within problems, the comparison that matters when choosing among one problem's prefixes. We also report each signal's Spearman correlation with the Monte-Carlo value.

Pruning experiment. On the problems whose 4 solutions all run past the cut, a signal keeps the 2 prefixes it ranks highest. We compare the mean value of the kept prefixes with the mean of all 4, which is what a random choice keeps in expectation. We also compare it with an oracle that ranks by half of the continuations and is scored on the other half. There are 95% bootstrap intervals over problems. We also evaluate the rule of the withdrawn v3 paper: prune a prefix when the judge's $p(\text{yes}) < 0.3$. For it we report how much it prunes, how many pruned prefixes were dead (value 0), and how many correct continuations it throws away.

4 Tree GRPO

Each prompt's group of 8 leaves grows from $R = 4$ sampled roots of 128 tokens. $K = 2$ of them are continued, with 4 samples each, from the exact token prefix. `tree_kev` keeps the two roots with the highest Kev-4B *on-track* score; `tree_random` keeps two at random. In distribution, `tree_random` equals sampling two roots

	128 tokens			256 tokens			512 tokens		
signal	AUROC	within	ρ	AUROC	within	ρ	AUROC	within	ρ
Kev-4B *on track*	0.76	0.61	0.53	0.77	0.64	0.52	0.77	0.65	0.46
Kev-4B *correct*	0.70	0.59	0.40	0.75	0.59	0.48	0.77	0.58	0.47
AR *on track*	0.77	0.63	0.55	0.79	0.64	0.56	0.79	0.67	0.50
policy log-prob.	0.67	0.64	0.34	0.72	0.66	0.42	0.73	0.64	0.39

Table 1: How well each signal predicts that a continuation of the prefix ends correct, by prefix length. AUROC pooled over all prefixes and within problems (pairs of continuations of the same problem with different outcomes), and the Spearman correlation ρ with the prefix's Monte-Carlo value. 500 validation problems, 4 solutions each, 4 continuations per prefix.

only. A root that has already ended is a complete solution, and if kept its 4 leaves are copies of it. Leaves are trained like Part II's flat completions: the full root and continuation tokens, the verifier's reward, and the group baseline over the prompt's 8 leaves. Everything else is Part II's `rlvr` setup. Every tenth step, the roots that were not kept are continued too, for the log only. This measures what the selection gained on that step against keeping all roots, and against the best two in hindsight.

Two effects pull in opposite directions. Selecting roots by a judge raises the share of correct leaves when the judge is right, which gives GRPO more groups with a learning signal. But it also makes the kept roots no longer samples of the policy, so the gradient is no longer an unbiased estimate of the policy gradient. Comparing `tree_kev` with `tree_random` (same tree shape, unbiased selection) isolates the selection, and comparing `tree_random` with Part II's `rlvr` isolates the tree shape.

5 Can a judge tell a promising prefix?

The base policy's 500×4 solutions gave 4,515 prefixes and 18,060 verified continuations, of which 37.8% are correct. A prefix's value spreads over the whole range. At 128 tokens the mean value is 0.41 and 41% of the prefixes are dead (no correct continuation). At 512 tokens these are 0.29 and 61%, because the solutions that are still running by then are the harder ones. So there is something to choose: among one problem's prefixes, some are worth continuing and some are not.

All signals carry information, none separates a problem's prefixes well. Table 1 gives each signal's AUROC for a continuation ending correct. Pooled over problems, every signal is clearly informative (0.67–0.77 at 128 tokens). Most of that is the difficulty of the problem, which every signal partly sees. Within problems, the comparison that matters for choosing which prefixes to continue, the AUROCs fall to 0.59–0.64 at 128 tokens and reach at most 0.67 at 512 tokens. Kev-4B's *on track* question does not do better than the AR judge of the same size asked the same question (within problems 0.64 against 0.64 at 256 tokens, a difference of -0.007 $[-0.033, 0.020]$; 95% intervals from resampling problems). Nor does it beat the policy's own mean token log-probability, which costs nothing (0.66; Kev-4B minus log-probability -0.021 $[-0.055, 0.014]$). Pooled over problems both judges are ahead of the log-probability (+0.053 $[0.026, 0.080]$ for Kev-4B), so what the judges add is mostly a sense of how hard the problem is, which does not help to choose among one problem's prefixes. Asking Kev-4B Part II's *correct* question about an unfinished solution works worse than asking whether it is on track.

Selecting prefixes helps a little, and any signal will do. On the problems whose four solutions all run past the cut, keeping the two prefixes a signal ranks highest raises the mean value of what is continued (Table 2). At 128 tokens Kev-4B's *on track* gains +0.046 $[0.034, 0.058]$ over the 0.39 of keeping all four. The AR judge

keep 2 of 4 by	128 tokens	256 tokens	512 tokens
problems	401	253	93
value of all 4	0.386	0.324	0.165
Kev-4B *on track*	+0.046 [0.034, 0.058]	+0.064 [0.047, 0.082]	+0.040 [0.015, 0.067]
Kev-4B *correct*	+0.040 [0.028, 0.052]	+0.043 [0.026, 0.060]	+0.021 [−0.003, 0.046]
AR *on track*	+0.045 [0.033, 0.057]	+0.069 [0.053, 0.085]	+0.048 [0.022, 0.075]
policy log-prob.	+0.052 [0.041, 0.063]	+0.065 [0.048, 0.083]	+0.040 [0.014, 0.067]
oracle (split half)	+0.048 [0.033, 0.063]	+0.098 [0.080, 0.117]	+0.059 [0.034, 0.086]

Table 2: Mean value of the 2 prefixes (of 4) a signal keeps, minus the mean value of all 4, on the problems whose 4 solutions all run past the cut. 95% bootstrap intervals over problems. The oracle ranks by half of the continuations and is scored on the other half.

	128 tokens			256 tokens			512 tokens		
signal	pruned	dead	lost	pruned	dead	lost	pruned	dead	lost
Kev-4B *on track*	1.6%	84%	0.2%	2.1%	85%	0.3%	1.5%	87%	0.3%
Kev-4B *correct*	12.1%	61%	6.4%	12.1%	72%	4.2%	17.1%	76%	8.2%
AR *on track*	4.6%	77%	1.2%	8.0%	78%	1.7%	14.4%	87%	3.9%

Table 3: The v3 rule, prune when $p(\text{yes}) < 0.3$: the fraction of prefixes pruned, the fraction of the pruned that were dead, and the fraction of all correct continuations that the pruned prefixes would have produced.

gains +0.045 and the policy's log-probability +0.052. The best gains are at 256 tokens, +0.064 for Kev-4B, +0.069 for the AR judge and +0.065 for the log-probability. The oracle, which ranks by two of the four continuations and is scored on the other two, gains +0.098 there. With four continuations per prefix the value itself is a noisy estimate, so the oracle's gain underestimates what perfect knowledge would give. At 128 tokens the signals gain about as much as this oracle, at 256 tokens about two thirds of it. In absolute terms the gains are small: a few more correct continuations in a hundred.

The pruning rule of the withdrawn v3 paper rarely fires. v3 pruned a prefix when the judge's $p(\text{yes})$ was below 0.3 (Table 3). Kev-4B's *on track* probability is almost never that low. The rule prunes 1.6% of the prefixes at 128 tokens. 84% of those are dead, but they are only 3% of all dead prefixes. With the *correct* question the rule prunes more (12.1%), with fewer dead prefixes among the pruned (61%), and it throws away 6.4% of all correct continuations. A fixed threshold on an uncalibrated probability is not a usable pruning rule here; ranking within a problem is.

6 Training with tree rollouts

Both tree conditions ran with 2 seeds of 200 steps, everything else as in Part II's `rlvr`, which is the flat reference (2 seeds). Table 5 gives the final policies' accuracies, Table 6 the paired differences, Table 4 what happened inside the trees during training, and Figure 1 the training curves. The intervals resample problems and do not include the variation between seeds.

The judge's choice gained little over a random one. Every tenth step the roots that were not kept are continued too, so we know what keeping the chosen two gained over keeping all four (Table 4). Under `tree_kev` the leaves of the two roots Kev-4B ranked highest were correct +1.4 [0.7, 2.1] points more often than the leaves of all four roots, over 40 audited steps. Under `tree_random`, whose choice gains nothing in

		roots	audit, every 10th step: all four roots continued					tokens
condition	seeds	ended	AUROC	within prompt	value	gain, kept 2	gain, best 2	per step (k)
`tree_random`	2	1.8%	0.76	0.57	60.8	+0.2	+8.1	63.0
`tree_kev`	2	1.3%	0.75	0.56	59.5	+1.4	+8.1	82.0

Table 4: Inside the trees during training (mean over steps and seeds): share of roots that had already ended within 128 tokens; on the audited steps (every tenth), where all four roots of a prompt are continued, the AUROC of Kev-4B's *on track* score of a root for its leaves' verdicts, pooled and within prompt, the leaf accuracy of all four roots (percent), and what keeping the chosen two, or the best two in hindsight, gained over it (points); tokens generated per step (thousands, last 20 steps), not counting the audit's continuations.

condition	seeds	MATH-500	GSM8K	AMC23	AIME24	AIME25
base model	–	76.5	87.9	50.6	10.0	10.9
`rlvr`	2	82.9 ± 0.1	93.2 ± 0.2	61.7 ± 2.7	21.5 ± 0.6	17.8 ± 1.3
`tree_random`	2	84.5 ± 3.0	93.1 ± 1.0	67.0 ± 5.0	27.2 ± 8.7	20.5 ± 6.0
`tree_kev`	2	84.9 ± 3.3	94.1 ± 0.9	65.8 ± 3.8	26.4 ± 7.8	20.0 ± 6.2

Table 5: Final policies: accuracy in percent, seed mean ± standard deviation over seeds. MATH-500 and GSM8K: greedy. AMC23, AIME24 and AIME25: mean over 16 samples per problem at temperature 0.6. `rlvr`: flat sampling (Part II).

expectation, the same number is +0.2 $[-0.3, 0.8]$; these intervals resample the audited steps. Keeping the two roots whose leaves turned out best would have gained +8.1 points, more than any ranking made before the leaves exist could, since the same four leaves per root both rank the roots and score them. On the audited steps Kev-4B's *on track* score of a root separates its correct from its incorrect leaves with an AUROC of 0.75 pooled over prompts, but only 0.56 $[0.54, 0.58]$ within a prompt, the comparison that decides which roots are kept (0.57 under `tree_random`). Offline, with the same structure (four prefixes of 128 tokens per problem, four continuations each), the within-problem AUROC was 0.61, and keeping the two best-ranked prefixes raised the share of continuations ending correct by +4.6 points (§5). The prefixes differ: offline they are the base policy's, in training the policy's own as it learns. Its roots lead to correct answers more and more often (under `tree_random`, 55.4% of the audited leaves were correct in the first half of training and 66.1% in the second, against 38.6% of the continuations of the base policy's prefixes offline), and the room for a choice, the best two in hindsight, shrank from +9.4 to +6.9 points.

Final accuracy: no effect of the judge's choice; the tree shape undecided. The two tree conditions end at the same accuracy. `tree_kev` minus `tree_random` is +0.4 $[-1.6, 2.3]$ points on MATH-500, −1.2 $[-3.4, 1.0]$ on AMC23 and −0.8 $[-3.5, 1.6]$ on AIME24 (Table 6); only the interval on GSM8K, +1.0 $[0.2, 1.9]$, excludes zero. Both tree conditions end above flat sampling in the seed mean (+1.6 and +2.0 points on MATH-500, +5.7 and +4.9 on AIME24, against `rlvr`), but not seed by seed: each has one seed that ends well above `rlvr` and one that ends at its level. On MATH-500 `tree_random` ends at 82.4% and 86.6%, `tree_kev` at 87.2% and 82.6%, and `rlvr` at 82.8% and 83.0%; on AIME24 at 21.0% and 33.3%, 31.9% and 20.8%, and 21.0% and 21.9%. The intervals over problems do not include this variation between seeds, so whether the tree shape itself helps, two seeds cannot tell. At step 100 neither tree condition was ahead on MATH-500 (80.5% and 82.4%, against 83.2% for `rlvr`).

One seed of `tree_kev` began to repeat itself. The training completions of `tree_kev`'s seed 2 came to repeat part of the solution after the first answer: 4% of them did in steps 0–24, 11% in steps 50–74, 41% in

minus tree_random	MATH-500	GSM8K	AMC23	AIME24	AIME25
rlvr	−1.6	+0.1	−5.2	−5.7	−2.7
	[−3.6, 0.3]	[−0.8, 0.9]	[−9.3, −1.6]	[−10.3, −1.5]	[−5.7, 0.1]
tree_kev	+0.4	+1.0	−1.2	−0.8	−0.5
	[−1.6, 2.3]	[0.2, 1.9]	[−3.4, 1.0]	[−3.5, 1.6]	[−3.5, 2.6]

Table 6: Paired differences to tree_random in percentage points: per problem, each condition's seed mean, averaged over problems, with a 95% bootstrap interval over problems. tree_kev minus tree_random isolates the judge's choice of roots; rlvr minus tree_random the tree shape.

		training completions, steps					final
condition	seed	0–24	50–74	100–124	150–174	175–199	MATH-500
tree_random	1	3%	1%	0%	0%	0%	0%
tree_random	2	6%	2%	7%	0%	0%	0%
tree_kev	1	3%	0%	0%	0%	0%	0%
tree_kev	2	4%	11%	41%	65%	59%	90%

Table 7: Share of completions that repeat part of the solution after the first boxed answer (a line of at least 20 characters that already occurred before it): the training completions of each run in five 25-step windows, and the final policy's greedy MATH-500 completions.

steps 100–124, 65% in steps 150–174 and 59% in steps 175–199 (Table 7; the share first exceeded a fifth in the window from step 75). They grew from 620 to 859 tokens on average; in the last 25 steps 51% of them were cut off at the length limit of 1024 tokens and the verifier accepted 63%. Seed 1 of tree_kev and both seeds of tree_random did not (at most 0%, 1% and 7% in any window from step 50 on). The final policy of seed 2 repeats after its first answer on 90% of the MATH-500 problems, that of seed 1 on 0%; it writes 931 tokens per solution against 567, and its accuracy, 82.6%, is that of rlvr. Several boxed answers alone are not repetition: seed 2 of tree_random gives more than one on 46% of the MATH-500 problems (seed 1: 3%) without repeating lines (0.2% of its lines repeat an earlier one, against 24.4% for seed 2 of tree_kev; Table 8). This is not the judge paying for repetition, as in Part II: the reward is the verifier's alone, and Kev-4B sees only a solution's first 128 tokens. Under the verifier, which reads the last answer, a repetition that restates the answer costs nothing. Whether the judge's choice of roots made the policy repeat itself, one seed of two cannot tell.

Cost. A tree samples four roots of at most 128 tokens per prompt and continues two of them four times each, so for the same completion length it generates fewer tokens than flat sampling: 63.0 thousand per step under tree_random, against 76.0 thousand for flat sampling (the last 20 steps). tree_kev generated 82.0 thousand, because the completions of its seed 2 grew long (above). Scoring the roots with Kev-4B on the shared backbone is the only extra cost of the judge's choice.

Evaluation noise. Across the 6 runs in this comparison, the base model's MATH-500 accuracy at step 0 ranges from 75.2% to 76.4%, and a run's accuracy changes by 1.6 points on average between its evaluations at steps 150 and 200. The final weights of each run are evaluated twice on MATH-500, both times greedily: at step 200 with the periodic evaluations, and in the final evaluation. The two results differ by 0.7 points on average and by up to 1.4 (6 runs). A difference of about a point between two single evaluations is within the noise of the evaluation itself.

			repeated	boxed answers		Kev-4B P_c	
condition	tokens	truncated	lines	several	none	right	wrong
`rlvr`	548	3.0%	0.9%	4%	2.9%	0.84	0.59
`tree_random`	551	5.0%	0.4%	25%	4.5%	0.86	0.58
`tree_kev`	749	7.4%	12.4%	56%	4.8%	0.89	0.61

Table 8: What the final policies write on MATH-500 (greedy, seed mean): completion tokens, share truncated at 2048 tokens, share of lines (of at least 20 characters) that repeat an earlier line of the same solution, share with more than one boxed answer or none (the verifier reads the last one), and Kev-4B's P_c for right and for wrong answers.

7 Discussion

Within a problem, the judge sees little of where a solution is going. Offline, Kev-4B's *on track* score separates the correct from the incorrect continuations of one problem's 128-token prefixes with an AUROC of 0.61, no better than the AR judge of the same size (0.63) or the policy's own log-probability (0.64), which costs nothing. Across problems every signal does better, because each partly sees how hard the problem is, and that does not help to choose among one problem's prefixes. In training, on the policy's own roots, the separation was weaker still (0.56 [0.54, 0.58]), and keeping Kev-4B's two roots of four raised the share of correct leaves by +1.4 [0.7, 2.1] points, against +0.2 for a random choice and +8.1 for the best two in hindsight. The gain was larger early in training (+2.2 points in the first half, +0.5 in the second, 20 audited steps each, without intervals), while the policy's roots came to lead to correct answers more often and the room for a choice shrank (best two in hindsight: +9.5 and +6.8 points).

A small gain at the roots did not change what the policy learned. With the same tree shape, keeping Kev-4B's roots instead of random ones left the final accuracy unchanged within the intervals over problems on four of the five benchmarks; on GSM8K it was +1.0 [0.2, 1.9] points higher, one interval of five over two seeds, which we do not count as an effect. The idea behind the judge's choice is that it gives GRPO more groups with mixed outcomes, the only groups with a learning signal. It did not: in the last 20 steps 16% of the groups had no correct leaf under `tree_kev` and 14% under `tree_random`, and 48% and 48% had only correct ones. The choice has a cost in principle: the kept roots are no longer samples of the policy, so the gradient is no longer an unbiased estimate of the policy gradient. We saw no effect of that on accuracy. One seed of `tree_kev` came to repeat its solution after the first answer, which neither seed of `tree_random` did. Its reward is the verifier's alone, which does not penalise a repetition that restates the answer, so this is not the judge paying for repetition; whether the choice of roots caused it, one seed of two cannot tell.

Whether the tree shape helps is open. Both tree conditions end above flat sampling in the seed mean, by +1.6 and +2.0 points on MATH-500 and +5.7 and +4.9 on AIME24, and `tree_random` generated fewer tokens per step than flat sampling (63.0 thousand against 76.0 thousand). But each tree condition has one seed well above `rlvr` and one at its level, and at step 100 neither was ahead. The intervals over problems do not include the variation between seeds, and with two seeds per condition the difference may be the seeds'. Testing the tree shape needs more seeds than we ran.

Limitations. One policy size (4B), one judge question (*on track*) asked of 128-token roots, one tree shape (four roots, two kept, four leaves each), 200 steps at the learning rate of Part II, and 2 seeds per tree condition (`rlvr`: 2). With more roots per prompt a judge has more to choose from, and so do its errors. The audit continues all roots only every tenth step, 20 audited steps per run; its intervals resample those steps, not seeds.

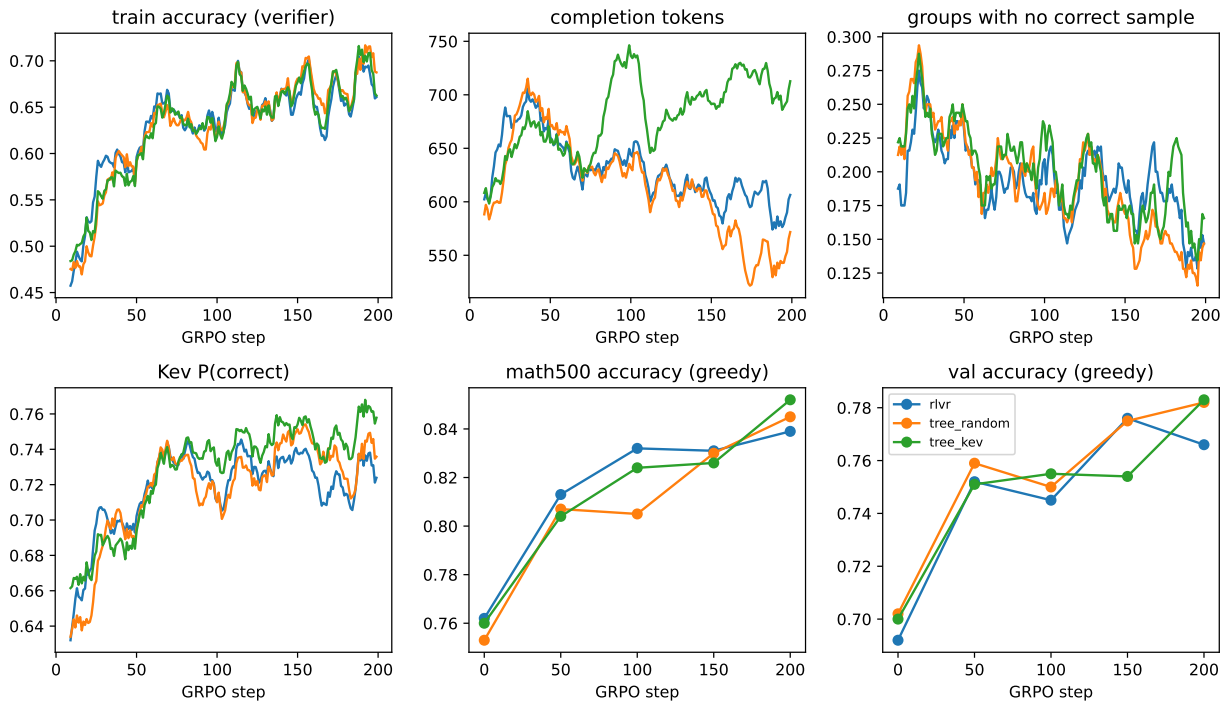


Figure 1: Training curves (seed mean): verifier accuracy, completion tokens, share of groups with no correct completion and Kev-4B's P_c on the training completions (10-step moving averages), and greedy accuracy on MATH-500 and the validation set every 50 steps.

The offline study used the base policy's prefixes, not the training policy's. We report tokens, not time: the runs shared GPUs with other experiments.

Relation to the earlier versions. Versions 1–3 of this paper reported pruning speed-ups, KV-cache reuse rates, per-node latencies and theorem-proving results (MiniF2F-Lean4, ProofNet) that were not measured (see the notice at the start and `CORRECTION.md`). None of these is measured here: we count tokens, not time, and ran no theorem proving. What we could test, we did. The pruning rule of v3, prune a prefix when the judge's $p(\text{yes}) < 0.3$, prunes 1.6% of the 128-token prefixes with the *on track* question and catches 3% of the dead ones (§5). Choosing which roots to continue by Kev-4B, the idea behind those versions, did not make GRPO learn better than a random choice of roots.

Reproducibility

Code, prompts, raw logs (every GRPO step and evaluation) and the prefix study's solutions, prefixes, continuations and judge scores are in the repository (`experiments/part3c_tree/`). The tables and every number in the text are generated from them by `dcjev/llm/prefix_study.py`, `dcjev/analysis/part2.py` and `dcjev/analysis/paper_numbers.py`.

References

[1] Zhenyu Hou, Ziniu Hu, Yujiang Li, Rui Lu, Jie Tang, and Yuxiao Dong. TreeRL: LLM reinforcement learning with on-policy tree search. In *Annual Meeting of the Association for Computational Linguistics*,

pages 12355–12369, 2025.

[2] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

[3] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025.

[4] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics*, 2024.